

Tools and Routing

```
In [1]: import os
import openai

from dotenv import load_dotenv, find_dotenv
_ = load_dotenv(find_dotenv()) # read local .env file
openai.api_key = os.environ['OPENAI_API_KEY']
```

```
In [2]: from langchain.agents import tool
```

```
In [3]: @tool
def search(query: str) -> str:
    """Search for weather online"""
    return "42f"
```

```
In [4]: search.name
```

```
Out[4]: 'search'
```

```
In [5]: search.description
```

```
Out[5]: 'search(query: str) -> str - Search for weather online'
```

```
In [6]: search.args
```

```
Out[6]: {'query': {'title': 'Query', 'type': 'string'}}
```

```
In [7]: from pydantic import BaseModel, Field
class SearchInput(BaseModel):
    query: str = Field(description="Thing to search for")
```

```
In [8]: @tool(args_schema=SearchInput)
def search(query: str) -> str:
    """Search for the weather online."""
    return "42f"
```

```
In [9]: search.args
```

```
Out[9]: {'query': {'title': 'Query',
                    'description': 'Thing to search for',
                    'type': 'string'}}
```

```
In [10]: search.run("sf")
```

```
Out[10]: '42f'
```

```
In [11]: import requests
from pydantic import BaseModel, Field
import datetime
```

```

# Define the input schema
class OpenMeteoInput(BaseModel):
    latitude: float = Field(..., description="Latitude of the location to fetch weather data for")
    longitude: float = Field(..., description="Longitude of the location to fetch weather data for")

@tool(args_schema=OpenMeteoInput)
def get_current_temperature(latitude: float, longitude: float) -> dict:
    """Fetch current temperature for given coordinates."""

    BASE_URL = "https://api.open-meteo.com/v1/forecast"

    # Parameters for the request
    params = {
        'latitude': latitude,
        'longitude': longitude,
        'hourly': 'temperature_2m',
        'forecast_days': 1,
    }

    # Make the request
    response = requests.get(BASE_URL, params=params)

    if response.status_code == 200:
        results = response.json()
    else:
        raise Exception(f"API Request failed with status code: {response.status_code}")

    current_utc_time = datetime.datetime.utcnow()
    time_list = [datetime.datetime.fromisoformat(time_str.replace('Z', '+00:00')) for time_str in results['hourly']['time']]
    temperature_list = results['hourly']['temperature_2m']

    closest_time_index = min(range(len(time_list)), key=lambda i: abs(time_list[i] - current_utc_time))
    current_temperature = temperature_list[closest_time_index]

    return f'The current temperature is {current_temperature}°C'

```

In [12]: `get_current_temperature.name`

Out[12]: `'get_current_temperature'`

In [13]: `get_current_temperature.description`

Out[13]: `'get_current_temperature(latitude: float, longitude: float) -> dict - Fetch current temperature for given coordinates.'`

In [14]: `get_current_temperature.args`

Out[14]: `{'latitude': {'title': 'Latitude', 'description': 'Latitude of the location to fetch weather data for', 'type': 'number'}, 'longitude': {'title': 'Longitude', 'description': 'Longitude of the location to fetch weather data for', 'type': 'number'}}`

```
In [15]: from langchain.tools.render import format_tool_to_openai_function
```

```
In [16]: format_tool_to_openai_function(get_current_temperature)
```

```
Out[16]: {'name': 'get_current_temperature',  
  'description': 'get_current_temperature(latitude: float, longitude: float)  
  -> dict - Fetch current temperature for given coordinates.',  
  'parameters': {'title': 'OpenMeteoInput',  
    'type': 'object',  
    'properties': {'latitude': {'title': 'Latitude',  
      'description': 'Latitude of the location to fetch weather data for',  
      'type': 'number'},  
      'longitude': {'title': 'Longitude',  
        'description': 'Longitude of the location to fetch weather data for',  
        'type': 'number'}}},  
    'required': ['latitude', 'longitude']}]}
```

```
In [17]: get_current_temperature({"latitude": 13, "longitude": 14})
```

```
Out[17]: 'The current temperature is 37.3°C'
```

```
In [18]: import wikipedia  
@tool  
def search_wikipedia(query: str) -> str:  
    """Run Wikipedia search and get page summaries."""  
    page_titles = wikipedia.search(query)  
    summaries = []  
    for page_title in page_titles[: 3]:  
        try:  
            wiki_page = wikipedia.page(title=page_title, auto_suggest=False)  
            summaries.append(f"Page: {page_title}\nSummary: {wiki_page.summa  
        except (  
            self.wiki_client.exceptions.PageError,  
            self.wiki_client.exceptions.DisambiguationError,  
        ):  
            pass  
    if not summaries:  
        return "No good Wikipedia Search Result was found"  
    return "\n\n".join(summaries)
```

```
In [19]: search_wikipedia.name
```

```
Out[19]: 'search_wikipedia'
```

```
In [20]: search_wikipedia.description
```

```
Out[20]: 'search_wikipedia(query: str) -> str - Run Wikipedia search and get page su  
mmaries.'
```

```
In [21]: format_tool_to_openai_function(search_wikipedia)
```

```
Out[21]: {'name': 'search_wikipedia',
  'description': 'search_wikipedia(query: str) -> str - Run Wikipedia search
and get page summaries.',
  'parameters': {'title': 'search_wikipediaSchemaSchema',
  'type': 'object',
  'properties': {'query': {'title': 'Query', 'type': 'string'}},
  'required': ['query']}
```

```
In [22]: search_wikipedia({"query": "langchain"})
```

```
Out[22]: 'Page: LangChain\nSummary: LangChain is a framework designed to simplify th
e creation of applications using large language models (LLMs). As a languag
e model integration framework, LangChain\'s use-cases largely overlap with
those of language models in general, including document analysis and summar
ization, chatbots, and code analysis.\n\nPage: OpenAI\nSummary: OpenAI is a
U.S. based artificial intelligence (AI) research organization founded in De
cember 2015, researching artificial intelligence with the goal of developin
g "safe and beneficial" artificial general intelligence, which it defines a
s "highly autonomous systems that outperform humans at most economically va
luable work".\nAs one of the leading organizations of the AI spring, it has
developed several large language models, advanced image generation models,
and previously, released open-source models. Its release of ChatGPT has bee
n credited with starting the AI spring.The organization consists of the non
-profit OpenAI, Inc. registered in Delaware and its for-profit subsidiary O
penAI Global, LLC. It was founded by Ilya Sutskever, Greg Brockman, Trevor
Blackwell, Vicki Cheung, Andrej Karpathy, Durk Kingma, Jessica Livingston,
John Schulman, Pamela Vagata, and Wojciech Zaremba, with Sam Altman and Elo
n Musk serving as the initial board members. Microsoft provided OpenAI Glob
al LLC with a $1 billion investment in 2019 and a $10 billion investment in
2023, with a significant portion of the investment in the form of computati
onal resources on Microsoft\'s Azure cloud service.On November 17, 2023, th
e board removed Altman as CEO, while Brockman was removed as chairman and t
hen resigned as president. Four days later, both returned after negotiatio
ns with the board, and most of the board members resigned. The new initial b
oard included former Salesforce co-CEO Bret Taylor as chairman. It was also
announced that Microsoft will have a non-voting board seat.\n\nPage: DataSt
ax\nSummary: DataStax, Inc. is a real-time data for AI company based in San
ta Clara, California. Its product Astra DB is a cloud database-as-a-service
based on Apache Cassandra. DataStax also offers DataStax Enterprise (DSE),
an on-premises database built on Apache Cassandra, and Astra Streaming, a m
essaging and event streaming cloud service based on Apache Pulsar. As of Ju
ne 2022, the company has roughly 800 customers distributed in over 50 count
ries.'
```

```
In [23]: from langchain.chains.openai_functions.openapi import openapi_spec_to_openai
from langchain.utilities.openapi import OpenAPISpec
```

```
In [24]: text = """
{
  "openapi": "3.0.0",
  "info": {
    "version": "1.0.0",
    "title": "Swagger Petstore",
    "license": {
      "name": "MIT"
```

```
    }
  },
  "servers": [
    {
      "url": "http://petstore.swagger.io/v1"
    }
  ],
  "paths": {
    "/pets": {
      "get": {
        "summary": "List all pets",
        "operationId": "listPets",
        "tags": [
          "pets"
        ],
        "parameters": [
          {
            "name": "limit",
            "in": "query",
            "description": "How many items to return at one time (max 100)",
            "required": false,
            "schema": {
              "type": "integer",
              "maximum": 100,
              "format": "int32"
            }
          }
        ],
        "responses": {
          "200": {
            "description": "A paged array of pets",
            "headers": {
              "x-next": {
                "description": "A link to the next page of responses",
                "schema": {
                  "type": "string"
                }
              }
            },
            "content": {
              "application/json": {
                "schema": {
                  "$ref": "#/components/schemas/Pets"
                }
              }
            }
          },
          "default": {
            "description": "unexpected error",
            "content": {
              "application/json": {
                "schema": {
                  "$ref": "#/components/schemas/Error"
                }
              }
            }
          }
        }
      }
    }
  }
}
```

```

    }
  },
  "post": {
    "summary": "Create a pet",
    "operationId": "createPets",
    "tags": [
      "pets"
    ],
    "responses": {
      "201": {
        "description": "Null response"
      },
      "default": {
        "description": "unexpected error",
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/Error"
            }
          }
        }
      }
    }
  }
},
"/pets/{petId}": {
  "get": {
    "summary": "Info for a specific pet",
    "operationId": "showPetById",
    "tags": [
      "pets"
    ],
    "parameters": [
      {
        "name": "petId",
        "in": "path",
        "required": true,
        "description": "The id of the pet to retrieve",
        "schema": {
          "type": "string"
        }
      }
    ],
    "responses": {
      "200": {
        "description": "Expected response to a valid request",
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/Pet"
            }
          }
        }
      }
    },
    "default": {

```

```

        "description": "unexpected error",
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/Error"
            }
          }
        }
      },
    },
  },
},
"components": {
  "schemas": {
    "Pet": {
      "type": "object",
      "required": [
        "id",
        "name"
      ],
      "properties": {
        "id": {
          "type": "integer",
          "format": "int64"
        },
        "name": {
          "type": "string"
        },
        "tag": {
          "type": "string"
        }
      }
    },
    "Pets": {
      "type": "array",
      "maxItems": 100,
      "items": {
        "$ref": "#/components/schemas/Pet"
      }
    },
    "Error": {
      "type": "object",
      "required": [
        "code",
        "message"
      ],
      "properties": {
        "code": {
          "type": "integer",
          "format": "int32"
        },
        "message": {
          "type": "string"
        }
      }
    }
  }
}

```

```

    }
  }
}
}

```

```
In [25]: spec = OpenAPISpec.from_text(text)
```

Attempting to load an OpenAPI 3.0.0 spec. This may result in degraded performance. Convert your OpenAPI spec to 3.1.* spec for better support.

```
In [26]: pet_openai_functions, pet_callable = openapi_spec_to_openai_fn(spec)
```

```
In [27]: pet_openai_functions
```

```
Out[27]: [{'name': 'listPets',
  'description': 'List all pets',
  'parameters': {'type': 'object',
    'properties': {'params': {'type': 'object',
      'properties': {'limit': {'type': 'integer',
        'maximum': 100.0,
        'schema_format': 'int32',
        'description': 'How many items to return at one time (max 100)'}}},
      'required': []}}}],
  {'name': 'createPets',
  'description': 'Create a pet',
  'parameters': {'type': 'object', 'properties': {}},
  {'name': 'showPetById',
  'description': 'Info for a specific pet',
  'parameters': {'type': 'object',
    'properties': {'path_params': {'type': 'object',
      'properties': {'petId': {'type': 'string',
        'description': 'The id of the pet to retrieve'}}},
      'required': ['petId']}]}
```

```
In [28]: from langchain.chat_models import ChatOpenAI
```

```
In [29]: model = ChatOpenAI(temperature=0).bind(functions=pet_openai_functions)
```

```
In [30]: model.invoke("what are three pets names")
```

```
Out[30]: AIMessage(content='', additional_kwargs={'function_call': {'name': 'listPets', 'arguments': '{"params":{"limit":3}}'}})
```

```
In [31]: model.invoke("tell me about pet with id 42")
```

```
Out[31]: AIMessage(content='', additional_kwargs={'function_call': {'name': 'showPetById', 'arguments': '{"path_params":{"petId":"42"}}'}})
```

Routing

In lesson 3, we show an example of function calling deciding between two candidate functions.

Given our tools above, let's format these as OpenAI functions and show this same behavior.

```
In [32]: functions = [
        format_tool_to_openai_function(f) for f in [
            search_wikipedia, get_current_temperature
        ]
    ]
    model = ChatOpenAI(temperature=0).bind(functions=functions)
```

```
In [33]: model.invoke("what is the weather in sf right now")
```

```
Out[33]: AIMessage(content='', additional_kwargs={'function_call': {'name': 'get_current_temperature', 'arguments': '{"latitude":37.7749,"longitude":-122.4194}'}})
```

```
In [34]: model.invoke("what is langchain")
```

```
Out[34]: AIMessage(content='', additional_kwargs={'function_call': {'name': 'search_wikipedia', 'arguments': '{"query":"Langchain}'}})
```

```
In [35]: from langchain.prompts import ChatPromptTemplate
        prompt = ChatPromptTemplate.from_messages([
            ("system", "You are helpful but sassy assistant"),
            ("user", "{input}"),
        ])
        chain = prompt | model
```

```
In [36]: chain.invoke({"input": "what is the weather in sf right now"})
```

```
Out[36]: AIMessage(content='', additional_kwargs={'function_call': {'name': 'get_current_temperature', 'arguments': '{"latitude":37.7749,"longitude":-122.4194}'}})
```

```
In [37]: from langchain.agents.output_parsers import OpenAIFunctionsAgentOutputParser
```

```
In [38]: chain = prompt | model | OpenAIFunctionsAgentOutputParser()
```

```
In [39]: result = chain.invoke({"input": "what is the weather in sf right now"})
```

```
In [40]: type(result)
```

```
Out[40]: langchain.schema.agent.AgentActionMessageLog
```

```
In [41]: result.tool
```

```
Out[41]: 'get_current_temperature'
```

```
In [42]: result.tool_input
```

```
Out[42]: {'latitude': 37.7749, 'longitude': -122.4194}
```

```
In [43]: get_current_temperature(result.tool_input)
```

```
Out[43]: 'The current temperature is 6.9°C'
```

```
In [44]: result = chain.invoke({"input": "hi!"})
```

```
In [45]: type(result)
```

```
Out[45]: langchain.schema.agent.AgentFinish
```

```
In [46]: result.return_values
```

```
Out[46]: {'output': 'Hello! How can I assist you today?'}
```

```
In [47]: from langchain.schema.agent import AgentFinish
def route(result):
    if isinstance(result, AgentFinish):
        return result.return_values['output']
    else:
        tools = {
            "search_wikipedia": search_wikipedia,
            "get_current_temperature": get_current_temperature,
        }
        return tools[result.tool].run(result.tool_input)
```

```
In [48]: chain = prompt | model | OpenAIFunctionsAgentOutputParser() | route
```

```
In [49]: result = chain.invoke({"input": "What is the weather in san francisco right
```

```
In [50]: result
```

```
Out[50]: 'The current temperature is 6.6°C'
```

```
In [51]: result = chain.invoke({"input": "What is langchain?"})
```

```
In [52]: result
```

```
Out[52]: 'Page: LangChain\nSummary: LangChain is a framework designed to simplify the creation of applications using large language models (LLMs). As a language model integration framework, LangChain's use-cases largely overlap with those of language models in general, including document analysis and summarization, chatbots, and code analysis.\n\nPage: OpenAI\nSummary: OpenAI is a U.S. based artificial intelligence (AI) research organization founded in December 2015, researching artificial intelligence with the goal of developing "safe and beneficial" artificial general intelligence, which it defines as "highly autonomous systems that outperform humans at most economically valuable work".\n\nAs one of the leading organizations of the AI spring, it has developed several large language models, advanced image generation models, and previously, released open-source models. Its release of ChatGPT has been credited with starting the AI spring.The organization consists of the non-profit OpenAI, Inc. registered in Delaware and its for-profit subsidiary OpenAI Global, LLC. It was founded by Ilya Sutskever, Greg Brockman, Trevor Blackwell, Vicki Cheung, Andrej Karpathy, Durk Kingma, Jessica Livingston, John Schulman, Pamela Vagata, and Wojciech Zaremba, with Sam Altman and Elon Musk serving as the initial board members. Microsoft provided OpenAI Global LLC with a $1 billion investment in 2019 and a $10 billion investment in 2023, with a significant portion of the investment in the form of computational resources on Microsoft's Azure cloud service.On November 17, 2023, the board removed Altman as CEO, while Brockman was removed as chairman and then resigned as president. Four days later, both returned after negotiations with the board, and most of the board members resigned. The new initial board included former Salesforce co-CEO Bret Taylor as chairman. It was also announced that Microsoft will have a non-voting board seat.\n\nPage: DataStax\nSummary: DataStax, Inc. is a real-time data for AI company based in Santa Clara, California. Its product Astra DB is a cloud database-as-a-service based on Apache Cassandra. DataStax also offers DataStax Enterprise (DSE), an on-premises database built on Apache Cassandra, and Astra Streaming, a messaging and event streaming cloud service based on Apache Pulsar. As of June 2022, the company has roughly 800 customers distributed in over 50 countries.'
```

```
In [53]: chain.invoke({"input": "hi!"})
```

```
Out[53]: 'Hello! How can I assist you today?'
```

```
In [ ]:
```

```
In [ ]:
```